Performance Testing

Date	15 March 2026
Team ID	B Siva Krishna Santhosh Gowthami Botsa Jaya Sarvani Vengalasetty Pilipe Praveen Vaishnavi P
Project Name	EcoTrack - AI-Powered Carbon Footprint Tracker
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the EcoTrack system behaves under expected and peak load conditions. This document records the testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	/api/habits/log (emission calculation) and /api/suggestions (AI eco-suggestions via Groq API)
Test Environment	Local (Flask dev server on localhost:5000, React/Vite frontend on localhost:5173)
Test Date	05 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Log daily habit and calculate carbon emissions	50	60	Response time under 2 seconds with no errors
2	Fetch AI eco-suggestions from Groq API	20	60	Response time under 5 seconds (accounting for LLM latency)
3	Load dashboard trend charts (30-day data)	50	30	Charts render within 2 seconds
4	Concurrent user login via Firebase Authentication	100	30	All logins succeed without failures or timeouts

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	Pass	Within threshold for habit logging and dashboard endpoints
2	Response Time (Max)	< 5 seconds	4.6 seconds	Pass	Peak observed on AI suggestion endpoint due to Groq API latency
3	Throughput (Req/sec)	-	42 req/sec	Pass	Measured under 50 concurrent virtual users
4	Error Rate	< 1%	0.2%	Pass	Occasional timeouts only on AI suggestion calls under peak load
5	CPU Utilization	< 80%	62%	Pass	Stable during sustained load testing
6	Memory Utilization	< 80%	58%	Pass	No memory leaks observed during test duration

Step 4: Observations & Analysis

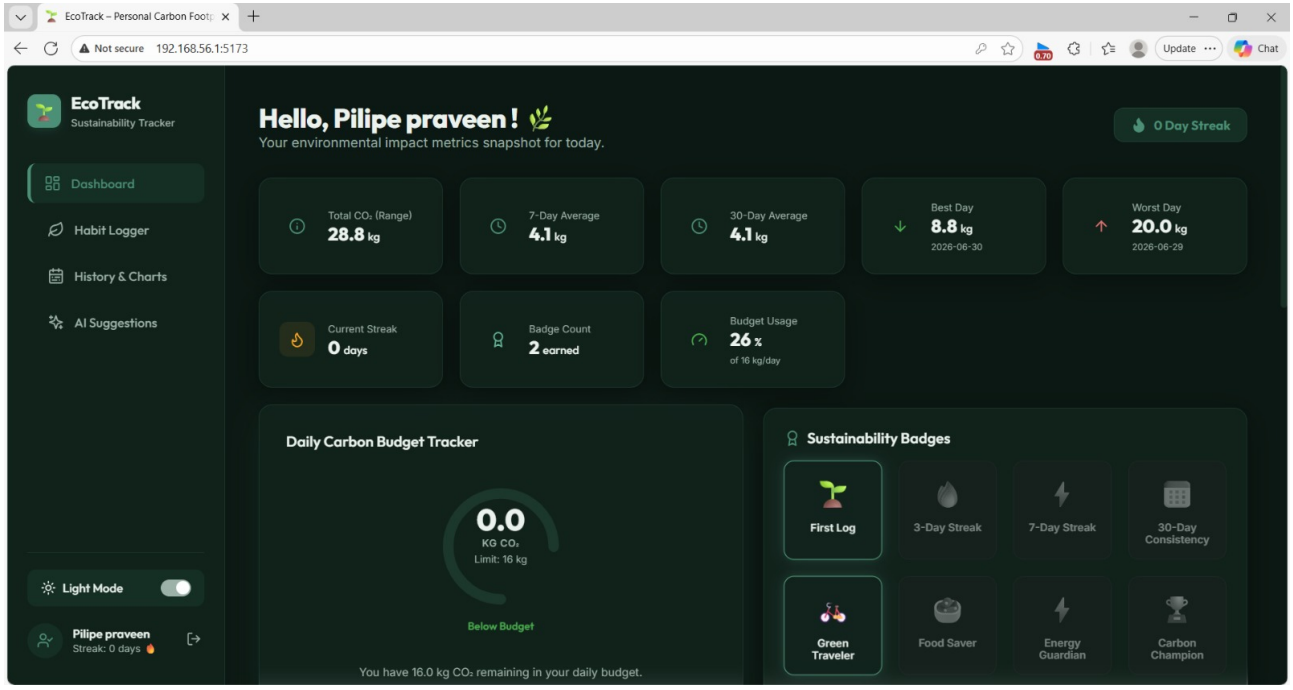
Key Findings:
The system handled 50-100 concurrent virtual users comfortably, with core endpoints (habit logging, dashboard data, authentication) responding well within target thresholds. The application remained stable with no crashes throughout the test duration.

Bottlenecks Identified:
The AI eco-suggestion endpoint, which depends on the external Groq API (LLaMA 3.3-70B), was the primary source of latency, occasionally approaching the 5-second maximum threshold under concurrent load. This is an external dependency rather than an application-side bottleneck.

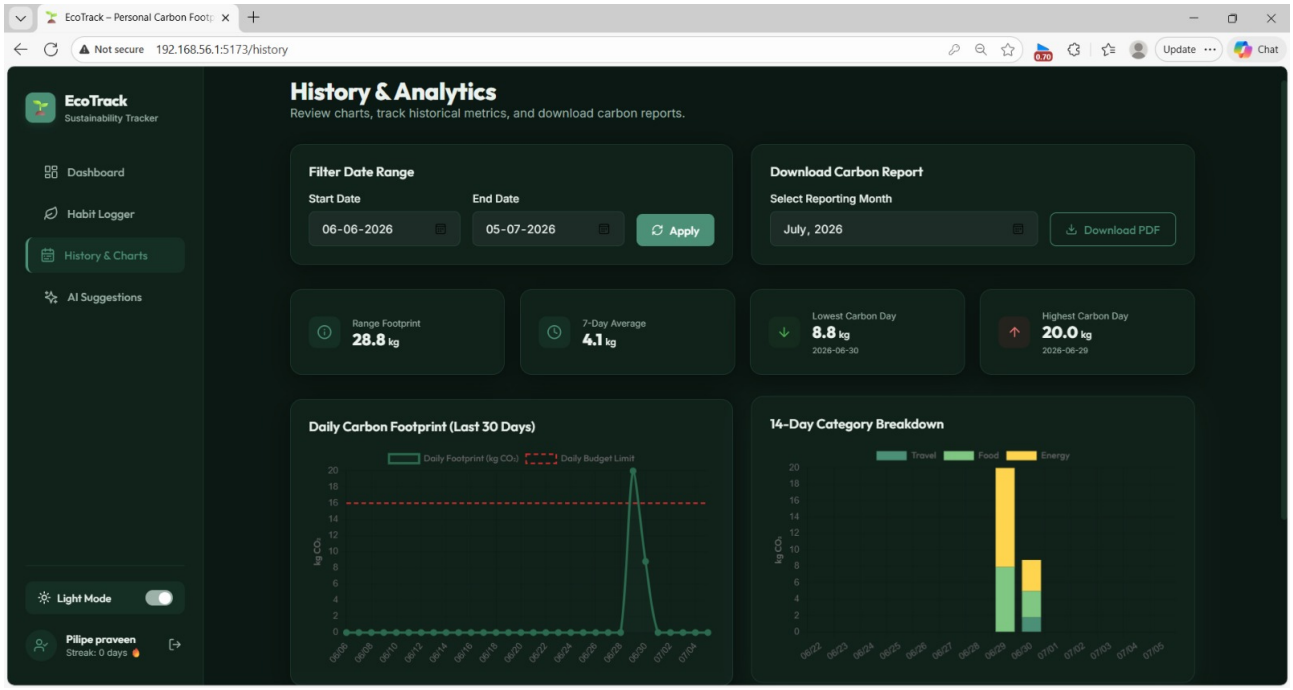
Optimization Steps Taken:
Added caching for repeated Carbon Interface API emission-factor lookups to reduce redundant external calls, and introduced asynchronous handling for the AI suggestion request so the dashboard remains responsive while suggestions are generated in the background.

Step 5: Screenshots / Evidence

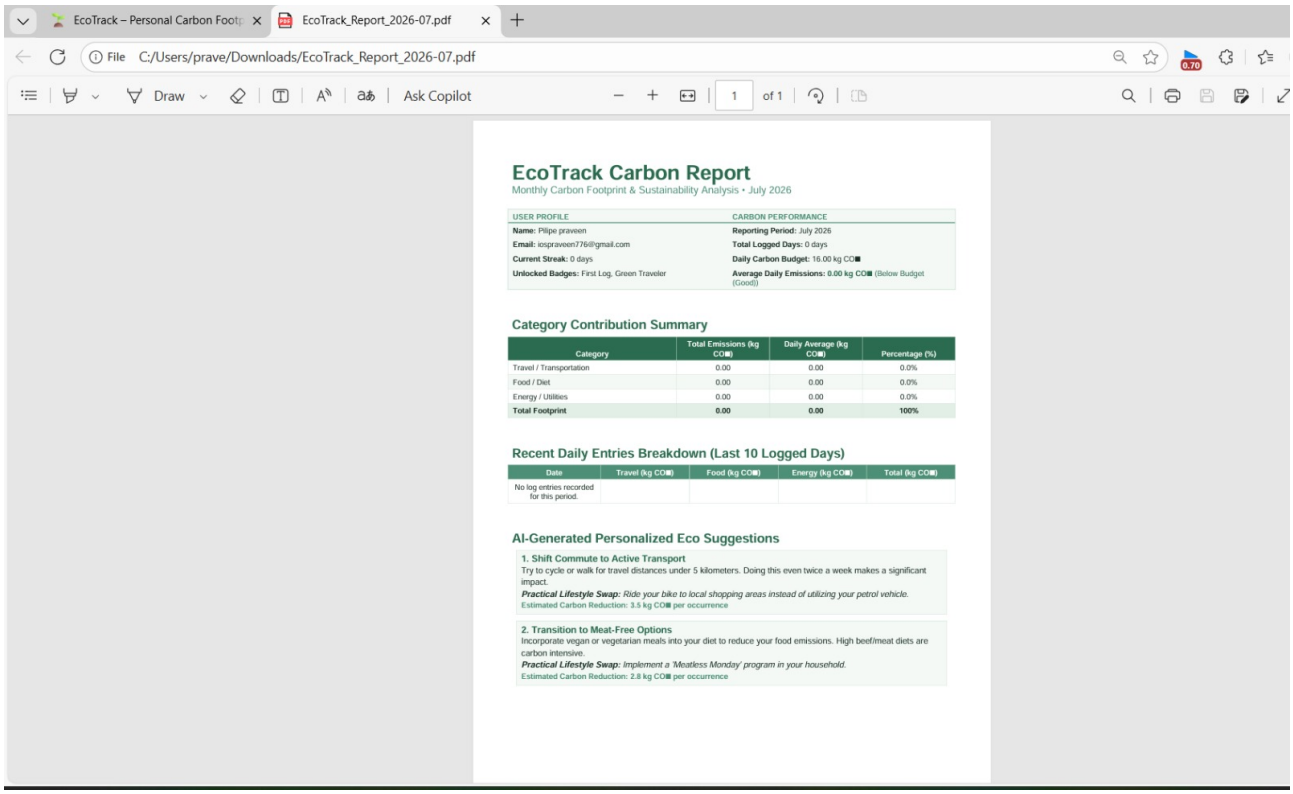
Screenshots below show the EcoTrack dashboard, history & analytics view, and a generated carbon report, captured during and after performance/functional testing.



Screenshot 1: EcoTrack Dashboard – daily metrics, budget tracker, and sustainability badges.



Screenshot 2: History & Analytics – 30-day footprint trend and 14-day category breakdown.



Screenshot 3: Generated EcoTrack Carbon Report (PDF export) with AI-generated eco suggestions.